

BÁO CÁO BÀI TẬP LỚN

SIMPLE OPERATING SYSTEM

Trần Nhật Bảo Anh – MSSV 2410155

Tháng 6 năm 2026

Contents

1	Tổng quan	2
2	Trả lời câu hỏi lý thuyết	2
2.1	Unified system-call interface	2
2.2	System call thực thi quá lâu	2
2.3	Lợi ích của các chính sách MLQ	2
2.4	Paging và continuous allocation	2
2.5	Kết hợp segmentation và paging	2
2.6	Lợi ích của N-level paging	2
2.7	Hậu quả khi thiếu synchronization	3
3	CPU Scheduling	3
3.1	Nguyên lý vận hành và cơ chế slot trong MLQ	3
3.2	Kết quả từ các testcase nhóm tự tạo	3
3.2.1	Kiểm thử độ ưu tiên và ngắt cường bức (sched_prio)	3
3.2.2	Kiểm thử hết slot chu kỳ (sched_out_of_slot)	4
3.2.3	Kiểm thử đa xử lý song song trên 2 CPU (sched_2_cpu)	5
3.3	Kết luận	5
4	System Call và phân tách User/Kernel	6
5	Memory Management	6
6	Paging 64-bit năm cấp	6
6.1	Address translation	6
6.2	Thống kê	7
7	Kiểm thử	7
8	Kết luận	7

1 Tổng quan

Dự án mô phỏng một hệ điều hành đa nhiệm gồm MLQ scheduler, unified system call, phân tách user/kernel space và paging 64-bit năm cấp. Mỗi CPU được mô phỏng bởi một thread. Scheduler queue, danh sách process đang chạy và physical frame free-list được bảo vệ bằng mutex.

Mỗi process sở hữu một `proc->mm`. Kernel không thay đổi một con trỏ `krnl->mm` dùng chung khi context switch, do đó nhiều CPU không thể vô tình thao tác trên address space của process khác.

2 Trả lời câu hỏi lý thuyết

2.1 Unified system-call interface

Ưu điểm là cung cấp một giao diện thống nhất, giảm coupling giữa chương trình user và implementation trong kernel, đồng thời tạo một điểm kiểm tra quyền truy cập tập trung. Nhược điểm là phát sinh overhead do trap, dispatch và validation; giao diện chung cũng có thể không biểu diễn tốt các đặc tính riêng của từng thiết bị.

2.2 System call thực thi quá lâu

Hệ điều hành dùng timer interrupt hoặc watchdog để phát hiện. Kernel có thể preempt tác vụ nếu kiến trúc cho phép, chuyển process sang trạng thái blocked khi chờ I/O, hoặc kết thúc process nếu vi phạm giới hạn. Kernel code phải tránh giữ lock trong thời gian dài để không chặn toàn hệ thống.

2.3 Lợi ích của các chính sách MLQ

Process có giá trị priority nhỏ được phục vụ trước. Số slot của mức i bằng $\text{MAX_PRIO} - i$, nên mức ưu tiên cao nhận nhiều CPU time hơn. FIFO trong từng queue tạo fairness giữa các process cùng mức. Khi các queue đang có process đều hết slot, scheduler reset slot để process priority thấp vẫn có cơ hội chạy.

2.4 Paging và continuous allocation

Continuous allocation dịch địa chỉ đơn giản nhưng gây external fragmentation và khó cấp phát vùng lớn. Paging loại bỏ external fragmentation và hỗ trợ swapping, đổi lại có internal fragmentation, chi phí page table và nhiều lần memory access khi dịch địa chỉ.

2.5 Kết hợp segmentation và paging

Segmentation phản ánh cấu trúc logic như code, data, heap và stack, thuận lợi cho protection. Paging quản lý physical memory theo frame cố định. Kết hợp hai cơ chế giữ được protection theo segment nhưng tránh external fragmentation khi nạp segment vào RAM.

2.6 Lợi ích của N-level paging

Không gian địa chỉ 64-bit rất thưa. Hierarchical paging chỉ cần tạo các nhánh được sử dụng thay vì một flat page table khổng lồ. Đánh đổi là mỗi translation phải truy cập nhiều bảng; TLB có thể giảm chi phí này.

2.7 Hậu quả khi thiếu synchronization

Hai CPU có thể lấy cùng PCB khỏi ready queue hoặc lấy cùng physical frame khỏi free-list. Kết quả là process chạy đồng thời trên hai CPU, page table bị ghi đè, double-free hoặc data corruption. Implementation dùng `queue_lock` cho scheduler và mutex cho frame allocator cùng các thao tác cập phát memory.

3 CPU Scheduling

Mục tiêu của phần này là mô phỏng bộ lập lịch CPU cho một hệ điều hành đa nhiệm dựa trên chính sách **Multi-Level Queue - MLQ** kết hợp cơ chế quản lý slot thời gian. Hệ thống hỗ trợ đa xử lý đối xứng (SMP) với nhiều lõi CPU ảo chạy đồng thời, có tính preemptive khi có tiến trình ưu tiên cao hơn xuất hiện.

3.1 Nguyên lý vận hành và cơ chế slot trong MLQ

- Hệ thống quản lý tối đa $MAX_PRIO = 140$ mức độ ưu tiên. Mỗi hàng đợi tương ứng với một mức ưu tiên cố định, giá trị độ ưu tiên càng nhỏ thì mức độ ưu tiên thực thi càng cao.
- Để tránh hiện tượng *starvation*, mỗi hàng đợi được cấp một số lượng slot giới hạn sử dụng CPU liên tiếp trong một chu kỳ điều phối theo công thức:

$$\text{slot}[\text{prio}] = MAX_PRIO - \text{prio} \quad (1)$$

- Bộ lập lịch sẽ duyệt hàng đợi từ mức ưu tiên cao nhất xuống thấp nhất ($0 \rightarrow 139$). Khi tiến trình được chọn, nó thực thi tối đa trong một khoảng thời gian *time_slice*.
- Nếu hàng đợi hết slot hoặc trống, hệ thống chuyển xuống hàng đợi tiếp theo. Khi tất cả các hàng đợi có tiến trình đều đã cạn kiệt slot, hệ thống kích hoạt cơ chế **Reset Slot** toàn cục để nạp lại giá trị ban đầu cho mảng slot.

3.2 Kết quả từ các testcase nhóm tự tạo

3.2.1 Kiểm thử độ ưu tiên và ngắt cưỡng bức (sched_prio)

Listing 1: Input cho sched_prio

```
2 1 2
0 s0 1
1 s1 0
```

Chú thích: *time_slice* = 2, thực thi trên 1 CPU, cấu hình chi tiết của 2 tiến trình bao gồm:

PID	Arrival	Burst	Priority
1	0	15	1
2	1	7	0

Table 1: Các process trong sched_prio

Khi *s0* đang thực thi dở dang chu kỳ lệnh của mình tại đầu Time slot 2, tiến trình *s1* với độ ưu tiên cao hơn ($PRIO = 0$) xuất hiện trong hàng đợi. Tại đầu Time slot 3, ngay khi kết

thức quantum thời gian của $s0$, bộ lập lịch thực hiện ngắt cường bức đưa $s0$ về trạng thái chờ và nhường quyền chiếm dụng CPU cho $s1$. Sau khi $s1$ hoàn thành toàn bộ công việc ở Time slot 10, $s0$ mới quay lại thực thi nốt phần Burst Time còn lại.

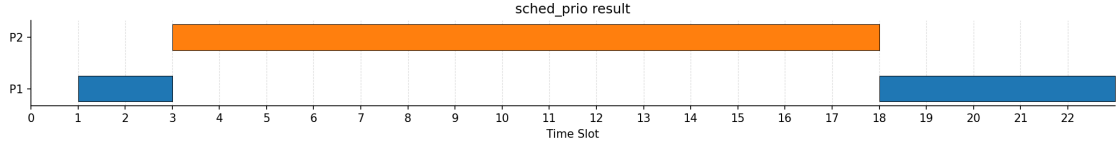


Figure 1: Biểu đồ Gantt kết quả cho test sched_prio

Kết quả từ biểu đồ Gantt ở trên cho thấy cơ chế ngắt cường bức hoạt động vô cùng chính xác tại thời điểm giao thoa giữa các *time_slice*. Luồng CPU không bị chiếm dụng vô thời hạn bởi tiến trình có độ ưu tiên thấp hơn khi có một tiến trình mang độ ưu tiên cao hơn sẵn sàng trong hệ thống. Đồng thời, log nạp tiến trình bất đồng bộ phản ánh đúng quan hệ song song giữa luồng CPU và luồng Loader.

3.2.2 Kiểm thử hết slot chu kỳ (sched_out_of_slot)

Listing 2: Input cho sched_out_of_slot

```
2 1 3
0 s0 139
1 s1 138
2 s2 137
```

Chú thích: time slice = 2, thực thi trên 1 CPU, cấu hình chi tiết của 3 tiến trình bao gồm:

PID	Arrival	Burst	Priority
1	0	15	139
2	1	7	138
3	2	12	137

Table 2: Các process trong sched_out_of_slot

Theo công thức phân bổ, số slot ban đầu được cấp lần lượt cho các hàng đợi là: $\text{slot}[137] = 3$, $\text{slot}[138] = 2$, và $\text{slot}[139] = 1$. Tiến trình $s0$ thực thi đầu tiên và tiêu thụ hết 1 slot của ‘Queue[139]’. Tại đầu Time slot 3, $s2$ xuất hiện và chiếm quyền thực thi nhờ độ ưu tiên cao nhất ($PRIO = 137$). Tiến trình $s2$ thực thi liên tục 3 *quantum* liên tiếp từ Time slot 3 đến đầu Time slot 9, làm cạn kiệt hoàn toàn 3 slot của ‘Queue[137]’.

Tại đầu Time slot 9, hệ thống luân chuyển quyền điều phối xuống cho tiến trình $s1$ thuộc hàng đợi kế tiếp. Tiến trình $s1$ thực thi tiêu tốn hết 2 slot từ Time slot 9 đến đầu Time slot 13. Lúc này, toàn bộ các tiến trình đang tồn tại đều đã sử dụng sạch các slot được cấp, hệ thống lập tức kích hoạt cơ chế **Reset Slot** toàn cục để nạp lại giá trị ban đầu, đưa $s2$ trở lại thực thi và giải phóng bộ nhớ ảo động (*MM64 dump*) khi kết thúc. Kịch bản này chứng minh logic chuyển đổi trạng thái (*Stateful transition*) của bộ lập lịch hoạt động hoàn toàn chính xác theo đặc tả hệ thống.

Kết quả từ biểu đồ Gantt ở trên cho thấy bộ lập lịch đã thực hiện việc theo dõi và khấu trừ giá trị ‘slot’ của từng hàng đợi một cách hiệu quả. Hiện tượng một tiến trình có độ ưu tiên cao chạy liên tục đúng số lần giới hạn rồi nhường quyền, và chu kỳ reset nạp lại năng lượng diễn ra tự động khi toàn bộ các hàng đợi cạn kiệt slot là minh chứng rõ nét cho thuật toán lập lịch MLQ được hiện thực hóa một cách chính xác.

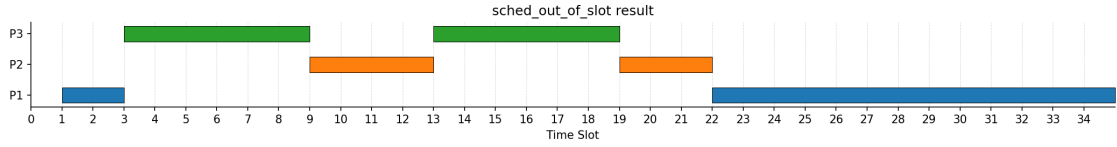


Figure 2: Biểu đồ Gantt kết quả cho test sched_out_of_slot

3.2.3 Kiểm thử đa xử lý song song trên 2 CPU (sched_2_cpu)

Listing 3: Input cho sched_out_of_slot

```
2 2 3
0 s0 138
0 s1 138
1 s2 137
```

Chú thích: time slice = 2, thực thi trên 2 CPUs, cấu hình chi tiết của 3 tiến trình bao gồm:

PID	Arrival	Burst	Priority
1	0	15	138
2	0	7	138
3	1	12	137

Table 3: Các process trong sched_2_cpu

Test này thể hiện rõ cách xử lý song song trong bộ lập lịch dùng 2 CPU:

Table 4: Bảng phân tích điều phối dòng thời gian trên 2 CPU

Thời gian (t)	CPU xử lý	Trạng thái điều phối và Chiếm dụng tài nguyên
1 → 3	CPU 0	Điều phối tiến trình s0 (PID 1) chạy trọn vẹn 1 Quantum.
2 → 4	CPU 1	Điều phối song song tiến trình s1 (PID 2) chạy đồng thời.
3 → 15	CPU 0	Ngắt s0, bốc siêu VIP s2 (PID 3) chạy liên tục qua các đợt reset slot.
4 → 15	CPU 1	Luân phiên điều phối luân hồi Round Robin giữa s0 và s1.

Cơ chế khóa đồng bộ pthread_mutex_lock đã bảo vệ nghiêm ngặt vùng nhớ dùng chung, tối đa 2 CPU bốc 2 tiến trình khác nhau, hoàn toàn không xảy ra xung đột Race Condition hay tranh chấp làm hỏng struct dữ liệu. Tại mốc thời gian 15, cả s2 và s1 cùng chạm vạch đích và giải phóng bộ nhớ cùng một lúc, để lại s0 một mình hoàn thành nốt chặng đường còn lại.

Kết quả từ biểu đồ Gantt ở trên cho thấy tính đúng đắn của giải thuật lập lịch đa lõi đối xứng. Hệ thống phân phối tối đa 2 tác vụ chạy song song trên 2 lõi độc lập, đảm bảo hiệu suất nạp tải của bộ vi xử lý phần cứng ảo mà không gây ra bất kỳ trạng thái Deadlock hay tranh chấp đọc/ghi nào trên cấu trúc dữ liệu hàng đợi dùng chung nhờ cơ chế loại trừ tương hỗ.

3.3 Kết luận

Qua quá trình triển khai cấu trúc dữ liệu mảng và kiểm thử thông qua các kịch bản thực nghiệm từ đơn giản đến phức tạp, module bộ lập lịch CPU dựa trên chính sách Multi-Level Queue đã chứng minh được tính ổn định và chính xác cao.

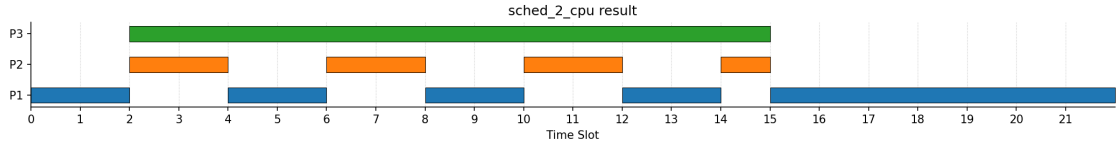


Figure 3: Biểu đồ trực quan Gantt chạy thực tế trên hệ thống 2 CPU (sched_2_cpu)

Kết quả thu được hoàn toàn tương thích và trùng khớp với các mẫu log đầu ra chuẩn được cung cấp trong cấu trúc mã nguồn của đồ án hệ thống. Cơ chế tính toán khấu trừ slot linh hoạt giúp tối ưu hóa hiệu năng, giảm thiểu tối đa hiện tượng starvation của các tiến trình có độ ưu tiên thấp, đồng thời cơ chế khóa đồng bộ hóa đảm bảo hệ thống bảo toàn tính nhất quán tuyệt đối trong môi trường xử lý đa luồng đa lõi song song.

4 System Call và phân tách User/Kernel

Các memory library wrapper ở user side chỉ dùng PCB để lấy PID và kernel handle, sau đó truyền scalar arguments qua syscall 17. Kernel gọi `find_proc_by_pid()` để duyệt `running_list`, `ready_queue` và MLQ queues dưới scheduler lock. PCB không được truyền như một syscall argument.

Raw physical read/write chỉ thành công nếu frame vật lý hiện đang được map trong page table của PID gọi syscall. Test `os_mem_protection` cho kết quả:

```
Kernel denied PID 1 physical read at 0x0
Kernel denied PID 1 physical write at 0x0
```

Kernel allocation sử dụng địa chỉ thuộc vùng `PAGING_KMEM_BASE` và yêu cầu một dãy physical frame liên tiếp. Kernel region có physical mapping riêng và không xuất hiện trong user page table.

5 Memory Management

Mỗi process có VMA, symbol region table và free-region list riêng. Khi `alloc` không tìm được free region đủ lớn, kernel tăng VMA limit, lấy frame từ RAM và map page. `free` đưa region trở lại free-region list để tái sử dụng. Read/write kiểm tra region ID và offset trước khi dịch địa chỉ.

6 Paging 64-bit năm cấp

6.1 Address translation

Địa chỉ canonical 57-bit được chia thành:

Phần	Bit	Số entry
PGD	56–48	512
P4D	47–39	512
PUD	38–30	512
PMD	29–21	512
PT	20–12	512
Offset	11–0	4096 byte/page theo sơ đồ địa chỉ

Implementation thực hiện walk thật theo chuỗi PGD -> P4D -> PUD -> PMD -> PT. Không gian mô phỏng hiện tại chỉ cần một nhánh sparse ở các cấp trên. Địa chỉ có các bit 63–57 không phải sign extension của bit 56 bị từ chối. Test `os_mm64_canonical`:

Sơ đồ địa chỉ 64-bit dùng offset 12 bit theo mô hình 4 KiB/page. Thiết bị physical-memory nhỏ trong simulator vẫn dùng frame granularity của framework gốc để các test RAM 1 KiB và 4 KiB tiếp tục kiểm tra được tình huống thiếu frame.

```
MM64 rejected non-canonical address 0x1000000000000000
```

6.2 Thống kê

Mỗi process dùng năm bảng, mỗi bảng 512 entry, mỗi entry 8 byte:

$$5 \times 512 \times 8 = 20480 \text{ byte.}$$

Mỗi page-table walk hợp lệ truy cập năm cấp, nên `accesses = 5 * walks`. Ví dụ output:

```
MM64 pid=1 va=0x0 indexes=[0,0,0,0,0]
walks=7 accesses=35 bytes=20480
```

7 Kiểm thử

Nhóm test	Kết quả	Mục tiêu
<code>sched, sched_0, sched_1</code>	PASS	MLQ và multi-CPU
<code>os_0/1/2_mlq_paging</code>	PASS	Scheduler kết hợp paging
<code>os_syscall, os_syscall_list, os_sc</code>	PASS	Syscall dispatch
<code>os_mm64_canonical</code>	PASS	Canonical-address validation
<code>os_mem_protection</code>	PASS	Từ chối raw physical access

Toàn bộ test được chạy bằng:

```
make clean
make all
make test
```

8 Kết luận

Hệ thống hoàn thành MLQ scheduling, unified syscall bằng PID, bảo vệ user/kernel space, cấp phát user/kernel memory, paging 64-bit năm cấp, canonical validation và thống kê page-table. Output đa CPU có thể khác thứ tự giữa các lần chạy nhưng mọi kết quả phải tuân theo priority, time slice và các điều kiện bảo vệ đã mô tả.